TP 02 — IoC et injection de dépendances (Spring Core)

> **Module** : IRT — Spring Boot 4 **Chapitre(s) du cours couvert(s)** : Chapitre 2 — Spring Boot & Spring Core (IoC / DI) **Durée estimée** : 4 h **Pré-requis** : TP 01 réalisé (savoir générer, lancer et tester un projet Spring Boot 4). JDK 25, IDE, `curl` /navigateur.

---

1. Objectifs pédagogiques

À la fin de ce TP, l'étudiant sera capable de :

- Distinguer **IoC** (Inversion of Control) et **DI** (Dependency Injection) et expliquer le rôle du **Spring IoC Container** (`ApplicationContext`).
- Déclarer des beans avec `@Component` et comprendre le **component scanning**.
- Injecter une dépendance par **constructeur** (recommandé), par **setter** et par **champ**, et justifier le choix.
- Résoudre l'ambiguïté `NoUniqueBeanDefinitionException` avec `@Qualifier` puis `@Primary` , et connaître leur priorité relative.
- Différer la création d'un bean avec `@Lazy` (et le mode global `spring.main.lazy-initialization`).
- Manipuler les **portées** (`singleton` , `prototype` , et les scopes web `request` , `session` , `application`).
- Exploiter le **cycle de vie** d'un bean avec `@PostConstruct` et `@PreDestroy` .
- Déclarer un bean à partir d'une classe non annotée via `@Configuration` + `@Bean` .

---

2. Contexte / mise en situation

Votre application de coaching sportif grandit. Chaque discipline (cricket, baseball, tennis, athlétisme, natation) a son **coach**, mais votre contrôleur ne doit **pas** dépendre d'une implémentation précise : il doit dépendre d'une **abstraction** (`Coach`). C'est exactement le rôle du conteneur Spring : créer les objets, gérer leurs dépendances et leur cycle de vie à votre place. Dans ce TP, vous construisez un « fil rouge » autour de l'interface `Coach` pour manipuler concrètement l'IoC et la DI.

---

3. Rappels de cours essentiels

- **IoC** : ce n'est plus l'objet qui crée ses dépendances, c'est un **conteneur externe** (le Spring IoC Container / `ApplicationContext`) qui les crée et les fournit.
- **DI** : le mécanisme concret par lequel Spring **injecte** les beans nécessaires dans les autres beans.
- **Bean** : objet géré par le conteneur (créé, configuré, détruit par lui).
- **Avantages** : couplage faible, code testable, configuration centralisée, gestion du cycle de vie.

3 modes d'injection : constructeur (préféré, dépendances `final` , objet toujours valide, facile à tester), setter (dépendances optionnelles/modifiables), champ (concis mais difficile à tester → **à éviter**).

Component scanning : par défaut, `@SpringBootApplication` scanne le package de la classe principale **et tous ses sous-packages**. L'id d'un bean = nom de la classe, 1re lettre en minuscule (`CricketCoach` → `cricketCoach`).

Tableau des annotations mobilisées :

Annotation	Rôle
`@Component`	Marque une classe comme bean (auto-détectée par component-scan)
`@Service` , `@Repository` , `@Controller` , `@RestController`	Spécialisations sémantiques de `@Component`
`@Autowired`	Demande l'injection d'une dépendance (constructeur / setter / champ)

Annotation	Rôle
`@Qualifier("id")`	Désigne précisément quel bean injecter quand plusieurs candidats
`@Primary`	Désigne un bean comme choix par défaut parmi plusieurs candidats
`@Lazy`	Diffère la création du bean au premier usage
`@Scope("...")`	Définit la portée (singleton, prototype, request, ...)
`@PostConstruct`	Méthode appelée après l'injection des dépendances
`@PreDestroy`	Méthode appelée avant destruction du bean
`@Configuration`	Classe contenant des définitions `@Bean`
`@Bean`	Méthode qui produit un bean géré par Spring

Règles d'or ambiguïté : un seul `@Primary` par type ; **`@Qualifier` est prioritaire sur `@Primary`** ; préférer `@Qualifier` quand on veut être explicite.

Cycle de vie : constructeur → injection des dépendances → `@PostConstruct` → ... usage ... → `@PreDestroy`.

> ⚠️ **Convention Spring Boot 4** : starter web = **`spring-boot-starter-webmvc`**. Imports du cycle de vie = **`jakarta.annotation.PostConstruct`** / **`jakarta.annotation.PreDestroy`** (jamais `javax.*`).

---

4. Mise en place de l'environnement

4.1 Générer le projet

Sur https://start.spring.io :

- **Group** : `com.ioc`
- **Artifact / Name** : `springcoredemo`
- **Package name** : `com.ioc.springcoredemo`
- **Java** : 25 · **Spring Boot** : 4.0.x · **Packaging** : Jar
- **Dependencies** : Spring Web (→ `spring-boot-starter-webmvc`).

4.2 `pom.xml` clé

```
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.0.0</version>
</parent>

<properties>
    <java.version>25</java.version>
</properties>

<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-webmvc</artifactId>
    </dependency>
</dependencies>
```

4.3 Organisation des packages (recommandée)

```
com.ioc.springcoredemo                ← classe @SpringBootApplication
com.ioc.springcoredemo.common         ← Coach + implémentations
com.ioc.springcoredemo.rest           ← DemoController
com.ioc.springcoredemo.config         ← SportConfig (@Configuration)
```

Tous ces sous-packages sont scannés automatiquement.

---

5. Travail à réaliser

> Enchaînez les exercices **dans l'ordre**. À chaque étape, lancez l'app et **testez l'endpoint** indiqué.

Exercice 5.1 — Interface `Coach`, un bean, injection par constructeur

Énoncé. Créez l'abstraction `Coach`, une implémentation `CricketCoach` marquée comme bean, et un contrôleur qui reçoit un `Coach` **par constructeur**.

Consignes.

1. Dans le package `common`, l'interface :

```
package com.ioc.springcoredemo.common;

public interface Coach {
    String getDailyWorkout();
}
```

2. Une implémentation annotée `@Component` :

```
package com.ioc.springcoredemo.common;

import org.springframework.stereotype.Component;

@Component
public class CricketCoach implements Coach {
    @Override
    public String getDailyWorkout() {
        return "Practice fast bowling for 15 minutes";
    }
}
```

3. Le contrôleur (package `rest`), à compléter :

```java
package com.ioc.springcoredemo.rest;

import com.ioc.springcoredemo.common.Coach;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class DemoController {

    private Coach myCoach;

    @Autowired
    public DemoController(Coach theCoach) {
        myCoach = theCoach;
    }

    @GetMapping("/dailyworkout")
    public String getDailyWorkout() {
        // TODO : déléguer à myCoach
        return /* ... */ ;
    }
}
```

Comment tester.

```
curl http://localhost:8080/dailyworkout
```

Résultat attendu : `Practice fast bowling for 15 minutes`

> **À noter** : depuis Spring 4.3, `@Autowired` est **optionnel** s'il n'y a qu'un seul constructeur. Essayez de le retirer et vérifiez que ça marche toujours.

Exercice 5.2 — Provoquer `NoUniqueBeanDefinitionException`

Énoncé. Ajoutez plusieurs implémentations de `Coach` pour créer une ambiguïté volontaire.

Consignes. Dans le package `common`, créez trois nouvelles implémentations **toutes** annotées `@Component` :

```
@Component
public class BaseballCoach implements Coach {
    @Override public String getDailyWorkout() { return "Spend 30 minutes on batting prac
}

@Component
public class TennisCoach implements Coach {
    @Override public String getDailyWorkout() { return "Practice your backhand volley";
}

@Component
public class TrackCoach implements Coach {
    @Override public String getDailyWorkout() { return "Run a hard 5k!"; }
}
```

Comment tester. Relancez l'application. **Résultat attendu** : le démarrage **échoue** avec une erreur du type :

```
NoUniqueBeanDefinitionException: No qualifying bean of type
'...Coach' available: expected single matching bean but found 4:
baseballCoach, cricketCoach, tennisCoach, trackCoach
```

Spring ne sait **plus** lequel injecter. Notez le message : les 4 ids de beans y figurent.

Exercice 5.3 — Résoudre l'ambiguïté avec `@Qualifier`

Énoncé. Indiquez explicitement le bean voulu.

Consignes. Modifiez le constructeur de `DemoController` :

```
import org.springframework.beans.factory.annotation.Qualifier;

@Autowired
public DemoController(@Qualifier("cricketCoach") Coach theCoach) {
    myCoach = theCoach;
}
```

> L'id par défaut d'un bean = nom de la classe, 1re lettre minuscule → `cricketCoach`. On peut aussi forcer un id : `@Component("monCoach")`.

Comment tester.

```
curl http://localhost:8080/dailyworkout
```

Résultat attendu : `Practice fast bowling for 15 minutes`. Changez le qualifier en `"tennisCoach"` et vérifiez que la réponse change.

Exercice 5.4 — Résoudre l'ambiguïté avec `@Primary`

Énoncé. Résolvez la même ambiguïté **sans** qualifier, en désignant un candidat par défaut.

Consignes.

1. Retirez temporairement le `@Qualifier` du constructeur (laissez `Coach theCoach`).
2. Marquez **un seul** coach comme prioritaire :

```
@Component
@Primary
public class TrackCoach implements Coach {
    @Override public String getDailyWorkout() { return "Run a hard 5k!"; }
}
```

Comment tester.

```
curl http://localhost:8080/dailyworkout
```

Résultat attendu : `Run a hard 5k!` (c'est `TrackCoach` qui est choisi).

Expérience à mener (priorité). Remettez **à la fois** `@Primary` sur `TrackCoach` **et** `@Qualifier("cricketCoach")` sur le constructeur. Quelle réponse obtenez-vous ? Concluez sur la règle « **`@Qualifier` gagne sur `@Primary`** ».

> **Piège** : mettez `@Primary` sur **deux** coachs et observez l'erreur au démarrage (plus d'un bean primaire).

Exercice 5.5 — Initialisation paresseuse (`@Lazy`)

Énoncé. Observez le moment de création des beans, puis rendez-les paresseux.

Consignes.

1. Ajoutez un **constructeur avec trace** dans chaque coach pour voir quand il est instancié :

```
@Component
public class TennisCoach implements Coach {
    public TennisCoach() {
        System.out.println(">> Constructeur : " + getClass().getSimpleName());
    }
    @Override public String getDailyWorkout() { return "Practice your backhand volley";
}
```

(faites de même pour les autres coachs.) 2. Lancez l'app **sans rien demander** : par défaut (eager), **tous** les constructeurs s'exécutent au démarrage. 3. Ajoutez `@Lazy` sur `TennisCoach` (en plus de `@Component`). Assurez-vous que le contrôleur injecte un **autre** coach (via `@Qualifier` ou `@Primary`), pour que `TennisCoach` ne soit pas requis.

Comment tester.

- Au démarrage : le message du constructeur de `TennisCoach` **ne doit plus apparaître** (il est lazy et personne ne le demande).
- **Mode global** : ajoutez dans `application.properties` :

```
spring.main.lazy-initialization=true
```

Relancez : **aucun** constructeur de coach ne s'affiche au démarrage. Le coach injecté n'est créé qu'au **premier appel** de `/dailyworkout` .

À expliquer. Avantages (démarrage plus rapide, économie mémoire) et inconvénient majeur (les erreurs de configuration ne sont détectées **qu'à l'exécution**).

> Retirez `spring.main.lazy-initialization=true` avant l'exercice suivant.

Exercice 5.6 — Portées : singleton vs prototype (endpoint `/check`)

Énoncé. Injectez **deux fois** le même bean et comparez les références pour distinguer singleton et prototype.

Consignes.

1. Modifiez `DemoController` pour recevoir deux fois le même coach et ajoutez un endpoint `/check` :

```
private Coach myCoach;
private Coach anotherCoach;

@Autowired
public DemoController(
        @Qualifier("cricketCoach") Coach theCoach,
        @Qualifier("cricketCoach") Coach theAnotherCoach) {
    myCoach = theCoach;
    anotherCoach = theAnotherCoach;
}

@GetMapping("/check")
public String check() {
    return "Comparing beans: myCoach == anotherCoach, " + (myCoach == anotherCoach);
}
```

2. Par défaut, `CricketCoach` est **singleton**.

Comment tester (singleton).

```
curl http://localhost:8080/check
```

Résultat attendu : `Comparing beans: myCoach == anotherCoach, true` (une seule instance partagée).

3. Passez `CricketCoach` en **prototype** :

```
import org.springframework.beans.factory.config.ConfigurableBeanFactory;
import org.springframework.context.annotation.Scope;

@Component
@Scope(ConfigurableBeanFactory.SCOPE_PROTOTYPE)
public class CricketCoach implements Coach { /* ... */ }
```

Comment tester (prototype).

```
curl http://localhost:8080/check
```

Résultat attendu : `... false` (deux instances différentes).

Tableau des scopes à compléter dans votre rapport :

Scope	Description
`singleton` (défaut)	Une seule instance dans le conteneur
`prototype`	Nouvelle instance à chaque demande
`request`	Une instance par requête HTTP (web only)
`session`	Une instance par session HTTP (web only)
`application`	Une instance par `ServletContext` (web only)

Exercice 5.7 — Cycle de vie : `@PostConstruct` / `@PreDestroy`

Énoncé. Rendez visible l'ordre du cycle de vie d'un bean.

Consignes. Sur `CricketCoach` (repassé en singleton pour cet exercice — `@PreDestroy` **n'est pas** appelé sur les beans prototype) :

```
import jakarta.annotation.PostConstruct;
import jakarta.annotation.PreDestroy;

@Component
public class CricketCoach implements Coach {

    public CricketCoach() {
        System.out.println("In constructor: " + getClass().getSimpleName());
    }

    @PostConstruct
    public void doMyStartupStuff() {
        System.out.println("In doMyStartupStuff()");
    }

    @PreDestroy
    public void doMyCleanupStuff() {
        System.out.println("In doMyCleanupStuff()");
    }

    @Override
    public String getDailyWorkout() {
        return "Practice fast bowling for 15 minutes";
    }
}
```

Comment tester.

- **Au démarrage** : la console affiche d'abord `In constructor:` puis `In doMyStartupStuff()`.
- **À l'arrêt propre** de l'application (bouton Stop de l'IDE, ou `Ctrl+C` en ligne de commande) : `In doMyCleanupStuff()` s'affiche.

À vérifier. Repassez `CricketCoach` en `prototype` : `@PreDestroy` **ne s'exécute plus** (Spring ne suit pas les beans prototype après création). Notez-le dans votre rapport.

Exercice 5.8 — `@Configuration` + `@Bean` : le `SwimCoach`

Énoncé. Déclarez comme bean une classe **non annotée** `@Component` (cas d'une classe tierce non modifiable).

Consignes.

1. Créez `SwimCoach` **sans** `@Component` :

```
package com.ioc.springcoredemo.common;

public class SwimCoach implements Coach {
    public SwimCoach() {
        System.out.println("In constructor: " + getClass().getSimpleName());
    }
    @Override
    public String getDailyWorkout() {
        return "Swim 1000 meters as a warm up";
    }
}
```

2. Enregistrez-le via une classe de configuration (package `config`) avec un **id explicite** :

```
package com.ioc.springcoredemo.config;

import com.ioc.springcoredemo.common.Coach;
import com.ioc.springcoredemo.common.SwimCoach;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class SportConfig {

    @Bean("aquatic")
    public Coach swimCoach() {
        return new SwimCoach();
    }
}
```

3. Injectez-le côté contrôleur :

```
@Autowired
public DemoController(@Qualifier("aquatic") Coach theCoach) {
    myCoach = theCoach;
}
```

Comment tester.

```
curl http://localhost:8080/dailyworkout
```

Résultat attendu : `Swim 1000 meters as a warm up`.

À rédiger. Quand préférer `@Bean` à `@Component` ? (classe externe non modifiable ; création complexe / factory ; plusieurs instances configurées différemment du même type).

---

6. Questions de compréhension

1. Expliquez avec vos mots la différence entre **IoC** et **DI**. En quoi le conteneur Spring « inverse-t-il le contrôle » par rapport à un `new CricketCoach()` classique ?
2. Pourquoi l'injection **par constructeur** est-elle recommandée par rapport à l'injection par **champ** ? Citez au moins deux arguments (immuabilité, testabilité…).
3. Dans quelle situation précise obtient-on `NoUniqueBeanDefinitionException` ? Donnez **deux** façons différentes de la résoudre et indiquez laquelle est prioritaire si les deux sont présentes.
4. Comment Spring calcule-t-il l'**id par défaut** d'un bean ? Comment le forcer manuellement ?
5. Quels sont les **avantages et inconvénients** de l'initialisation paresseuse (`@Lazy`) ? Pourquoi « à utiliser avec parcimonie » ?
6. Un endpoint `/check` renvoie `true` puis, après une modification, `false`. Qu'avez-vous changé, et qu'est-ce que cela prouve sur la portée du bean ?
7. Donnez l'**ordre** exact des étapes du cycle de vie d'un bean singleton. Pourquoi `@PreDestroy` n'est-il pas appelé pour un bean `prototype` ?
8. Citez **trois** cas où l'on préfère `@Bean` (dans une `@Configuration`) plutôt que `@Component` directement sur la classe.

---

7. Livrables attendus

- Le projet `springcoredemo` (sans `target/`) ou un lien Git.
- Les fichiers : `Coach` , `CricketCoach` , `BaseballCoach` , `TennisCoach` , `TrackCoach` , `SwimCoach` , `SportConfig` , `DemoController` .

- **Captures** : la stacktrace `NoUniqueBeanDefinitionException` (5.2) ; les réponses de `/dailyworkout` avec `@Qualifier` puis `@Primary` ; les réponses `/check` **true** puis **false** ; la console montrant `In constructor` → `In doMyStartupStuff()` → `In doMyCleanupStuff()`.
- Un `REPONSES.md` traitant les **8 questions** de la section 6, incluant le tableau des scopes complété.

---

8. Barème indicatif (/20)

Critère	Points
5.1 Interface `Coach` + `@Component` + injection constructeur + `/dailyworkout`	3
5.2 Ambiguïté provoquée (`NoUniqueBeanDefinitionException`)	1
5.3 Résolution avec `@Qualifier`	2
5.4 Résolution avec `@Primary` + démonstration de la priorité Qualifier > Primary	2
5.5 `@Lazy` (bean + global) avec traces de construction	2
5.6 Scopes singleton vs prototype via `/check` (true/false)	3
5.7 `@PostConstruct` / `@PreDestroy` : ordre observable	2
5.8 `@Configuration` + `@Bean("aquatic")` (`SwimCoach`)	2
Réponses aux questions de compréhension	2
Qualité (organisation des packages, imports `jakarta.*` , propreté)	1
Total	**20**

---

9. Pour aller plus loin (bonus)

1. **Scope `request`** : créez un bean `@Scope(value = WebApplicationContext.SCOPE_REQUEST, proxyMode = ScopedProxyMode.TARGET_CLASS)` qui stocke un identifiant unique par requête, et exposez-le dans un endpoint. Vérifiez qu'il change à chaque appel HTTP.
2. **Injection d'une collection** : injectez `List<Coach>` dans le contrôleur et créez un endpoint `/allworkouts` qui liste les workouts de **tous** les coachs. Observez que Spring injecte automatiquement **tous** les beans du type.

3. **`@Bean` avec `initMethod` / `destroyMethod`** : reproduisez le comportement de `@PostConstruct` / `@PreDestroy` sur `SwimCoach` via les attributs `@Bean(initMethod = "...", destroyMethod = "...")`, sans annoter la classe.